

explain why the following calculate route blocks the rest of the incoming requests:

```
const express = require('express');
```

```
const app = express();
```

```
async function calculate() {  
  let sum = 0;
```

```
  for (let i = 0; i < 10_000_000_000; i++) {  
    sum += i;
```

Show more ▾

The key thing to understand is that `async` **does not automatically make code non-blocking**.

Your `/calculate` route looks asynchronous:

⌘ JavaScript



```
app.get('/calculate', async (req, res) => {  
  const calculatedData = await calculate();  
  res.send(`calculated data: ${calculatedData}`);  
});
```

but the work inside `calculate()` is still **CPU-bound synchronous JavaScript**:

⌘ JavaScript



```
async function calculate() {  
  let sum = 0;  
  
  for (let i = 0; i < 10_000_000_000; i++) {  
    sum += i;  
  }  
  
  return sum;  
}
```

What actually happens

When a request arrives at `/calculate` :

1. Express calls the route handler.
2. `calculate()` starts executing.
3. The `for` loop runs on the **main JavaScript thread**.
4. The loop occupies the event loop until it finishes.
5. Only after the loop completes does the promise resolve.
6. `await` resumes and sends the response.

The important part is that the loop runs **before the promise can resolve**.

Why `async` doesn't help

Consider this simpler example:

JavaScript

```
async function test() {  
  console.log('start');  
  
  for (let i = 0; i < 1_000_000_000; i++) {}  
  
  console.log('end');  
}
```

Calling:

JavaScript

```
test();  
console.log('after');
```

prints:

```
start  
end  
after
```

not:

```
start
after
end
```



because there is **no asynchronous operation inside the function**.

The `async` keyword merely makes the function return a Promise.

It's roughly equivalent to:

```
<> JavaScript
```



```
function calculate() {
  return Promise.resolve(
    // huge blocking calculation
  );
}
```

The calculation still happens immediately on the main thread.

Compare with a truly asynchronous operation

This doesn't block:

```
<> JavaScript
```



```
async function calculate() {
  const result = await fetch('https://api.example.com/data');
  return result.json();
}
```

Why?

Because:

1. Node starts the network request.
2. Control returns to the event loop.
3. Other requests can be processed.
4. When the response arrives, Node resumes the function.

The CPU isn't busy waiting.

What happens to other requests?

Imagine:

Request 1

GET /calculate



starts the loop.

While it's running:

Request 2

GET /home



arrives.

Node would like to execute:

<> JavaScript



```
app.get('/home', (req, res) => {  
  res.send('Performance example');  
});
```

but it cannot because the JavaScript thread is still busy executing:

<> JavaScript



```
for (let i = 0; i < 10_000_000_000; i++)
```

So Request 2 waits.

The same happens for:

GET /timer

GET /home

GET /calculate



Everything queues up until the loop finishes.

Why /timer also blocks

Your delay function is another example of CPU blocking:

</> JavaScript 

```
function delay(duration) {  
  const startTime = Date.now();  
  
  while(Date.now() - startTime < duration) {  
    // Event loop blocked  
  }  
}
```

This is called **busy waiting**.

During those 9 seconds:

</> JavaScript 

```
delay(9000);
```

the event loop cannot process:

- incoming HTTP requests
- timers
- callbacks
- promise continuations

Nothing else runs.

The event loop view

Normally:

Event Loop 

↓

Request A

↓

Request B

↓

Request C

Each callback runs quickly and returns.

With `/calculate` :

```
Event Loop
  ↓
Request A (/calculate)
  ↓
10 billion iterations
  ↓
10 billion iterations
  ↓
10 billion iterations
```



The event loop never gets a chance to move on.

How to make heavy calculations non-blocking

For CPU-intensive work, use:

1. Worker Threads (recommended)

JavaScript

```
const { Worker } = require('worker_threads');
```



The calculation runs on another thread, leaving the event loop free.

2. Child Processes

JavaScript

```
const { fork } = require('child_process');
```



The work runs in a separate Node process.

3. Break the work into chunks

JavaScript

```
function calculateChunked() {
  let i = 0;

  function chunk() {
    for (let j = 0; j < 1000000 && i < 10000000000; j++, i++) {
```



```

        // work
    }

    if (i < 10000000000) {
        setImmediate(chunk);
    }
}

chunk();
}

```

`setImmediate()` yields control back to the event loop between chunks.

Rule of thumb

A function is non-blocking **only if it yields control back to the event loop**.

These block:

</> JavaScript



```

while (...)
for (...)
JSON.parse(hugeString)
crypto.pbkdf2Sync(...)
fs.readFileSync(...)

```

These do not block:

</> JavaScript



```

await fetch(...)
await fs.promises.readFile(...)
setTimeout(...)
setImmediate(...)
crypto.pbkdf2(...)

```

So your `/calculate` route blocks because the expensive `for` loop runs entirely on Node's single JavaScript thread, and `async/await` does not move that computation off the event loop.

I need you to use the following code and show me how i can use each of the available modules and functions to show blocking and non-blocking working scenarios and provide a detailed explanation for each (I'll still use it as js code):

```
const express = require('express');
const { worker } = require('worker_threads');
const { fork } = require('child_process');
```

```
const app = express();
```

Show more ▾

This is actually a great learning setup because it lets you observe **all four major ways work happens in Node.js**:

1. Blocking synchronous work on the Event Loop
2. Non-blocking I/O using the OS/libuv
3. CPU work using the libuv Thread Pool
4. CPU work using Worker Threads or Child Processes

Let's build on your code and examine each scenario.

1. Blocking Example: Busy Wait

You already have:

<> JavaScript



```
function delay(duration) {
  const startTime = Date.now();

  while (Date.now() - startTime < duration) {
    // blocking
  }
}
```

Route:

</> JavaScript



```
app.get('/timer', (req, res) => {  
  delay(9000);  
  res.send('Ding ding ding!');  
});
```

What happens?

Request:

</> http



GET /timer

The event loop enters:

</> JavaScript



```
while (...) {}
```

and cannot leave until 9 seconds pass.

Timeline

t=0s /timer request arrives



Event Loop

```
| while(...) running |
```

t=2s /home request arrives

t=3s /home request arrives

t=5s /calculate request arrives

All requests waiting...

t=9s

while loop ends

/home processed

```
/home processed  
/calculate processed
```

Why?

JavaScript executes on a single thread.

While the loop is running:

```
</> JavaScript  
  
while (...) {}
```



the event loop cannot process:

- HTTP requests
- Promises
- Timers
- File reads

Everything waits.

2. Blocking Example: Huge CPU Calculation

Your code:

```
</> JavaScript  
  
async function calculate() {  
  let sum = 0;  
  
  for (let i = 0; i < 10_000_000_000; i++) {  
    sum += i;  
  }  
  
  return sum;  
}
```



Route:

```
</> JavaScript
```



```
app.get('/calculate', async (req, res) => {  
  const calculatedData = await calculate();  
  res.send(`calculated data: ${calculatedData}`);  
});
```

Why does it block?

Many people see:

JavaScript

```
async function calculate()
```

and think:

Runs in background

No.

The loop still runs immediately:

JavaScript

```
for (...)
```

on the Event Loop thread.

Actual execution

/calculate request

calculate()

↓

for loop starts

↓

10 billion iterations

↓

Promise resolves

↓

await resumes

↓
response sent

The event loop is trapped inside the loop.

3. Blocking Example: JSON Parsing

A real-world example many developers accidentally create:

```
</> JavaScript
app.get('/parse', (req, res) => {

  const hugeObject = new Array(10_000_000)
    .fill({name: "John"});

  const json = JSON.stringify(hugeObject);

  res.send('Done');
});
```

Why is this blocking?

Because:

```
</> JavaScript
JSON.stringify()
```

is synchronous.

Node must:

```
object
↓
serialize
↓
serialize
↓
serialize
```

↓
serialize

until complete.

No request can be processed during this time.

4. Non-Blocking Example: setTimeout

Route:

</> JavaScript



```
app.get('/timeout', (req, res) => {  
  
  setTimeout(() => {  
    res.send('Finished');  
  }, 9000);  
  
});
```

What happens?

Node registers a timer.

Then immediately returns control to the event loop.

Timeline

t=0

/timeout request

setTimeout registered

Event Loop free



Now:

</> http



GET /home

works instantly.

Why?

The callback is not executed now.

Only scheduled.

</> JavaScript



```
setTimeout(callback, 9000);
```

returns immediately.

5. Non-Blocking Example: setImmediate

Let's use your chunking function.

</> JavaScript



```
function calculateChunked() {  
  
    let i = 0;  
  
    function chunk() {  
  
        for (  
            let j = 0;  
            j < 1000000 && i < 100000000000;  
            j++, i++  
        ) {  
            // work  
        }  
  
        if (i < 100000000000) {  
            setImmediate(chunk);  
        }  
    }  
  
    chunk();  
}
```

Why doesn't this block?

The work is split into pieces.

Instead of:

```
10 billion iterations
```



we do:

```
1 million  
yield
```



```
1 million  
yield
```

```
1 million  
yield
```

Event Loop View

```
Chunk 1
```

```
↓
```

```
yield
```

```
/home request
```

```
↓
```

```
processed
```

```
Chunk 2
```

```
↓
```

```
yield
```

```
/home request
```

```
↓
```

```
processed
```



The server remains responsive.

6. Non-Blocking Network I/O

Example:

JavaScript



```
app.get('/users', async (req, res) => {  
  
  const response =  
    await fetch('https://jsonplaceholder.typicode.com/users');  
  
  const users =  
    await response.json();  
  
  res.json(users);  
  
});
```

What happens?

Node does:

Start HTTP request

↓

Continue processing requests

↓

Response arrives

↓

Resume async function



The event loop is never blocked waiting.

7. Non-Blocking File Reading

JavaScript



```
const fs = require('fs/promises');  
  
app.get('/file', async (req, res) => {  
  
  const data =  
    await fs.readFile('./large-file.txt', 'utf8');  
  
  res.send(data);  
  
});
```



```
});
```

Behind the scenes

Node hands the work to:

libuv Thread Pool



Event Loop



Thread Pool reads file



Event Loop continues serving requests



Thread finishes



Callback queued



8. CPU Work Using Thread Pool

Example:

<> JavaScript



```
const crypto = require('crypto');

app.get('/hash', (req, res) => {

  crypto.pbkdf2(
    'password',
    'salt',
    100000,
    512,
    'sha512',
    () => {
      res.send('Done');
    }
  );
});
```

```
});
```

Why doesn't this block?

Because:

```
</> JavaScript  
crypto.pbkdf2()
```



uses libuv's thread pool.

The CPU work happens on another thread.

Compare

Blocking:

```
</> JavaScript  
crypto.pbkdf2Sync(...)
```



Non-blocking:

```
</> JavaScript  
crypto.pbkdf2(...)
```



9. Worker Threads

Your import:

```
</> JavaScript  
const { Worker } = require('worker_threads');
```



Create:

</> JavaScript



```
// worker.js
```

```
let sum = 0;
```

```
for (let i = 0; i < 10_000_000_000; i++) {  
  sum += i;  
}
```

```
parentPort.postMessage(sum);
```

Route:

</> JavaScript



```
app.get('/worker-calc', (req, res) => {
```

```
  const worker =  
    new Worker('./worker.js');
```

```
  worker.on('message', result => {  
    res.send(`Result: ${result}`);  
  });
```

```
});
```

What happens?

Main Thread



↓

Create Worker

↓

Worker executes loop

↓

Main Thread keeps serving requests

Now /home remains fast.

Worker Thread Architecture

Main Thread

```
|  
├─ Express  
├─ Event Loop  
└─ Requests
```

Worker Thread

```
|  
└─ CPU Calculation
```



10. Child Process

Your import:

</> JavaScript

```
const { fork } = require('child_process');
```



Child:

</> JavaScript

```
// calculator.js
```

```
process.on('message', () => {  
  
  let sum = 0;  
  
  for (let i = 0; i < 10_000_000_000; i++) {  
    sum += i;  
  }  
  
  process.send(sum);  
  
});
```



Route:

</> JavaScript

```
app.get('/fork-calc', (req, res) => {  
  
  const child =  
    fork('./calculator.js');
```



```
child.send('start');

child.on('message', result => {
  res.send(`Result: ${result}`);
});

});
```

Difference from Worker Threads

Worker Thread:

Same process
Different thread
Shared memory possible



Child Process:

Completely separate process
Own memory
Own event loop

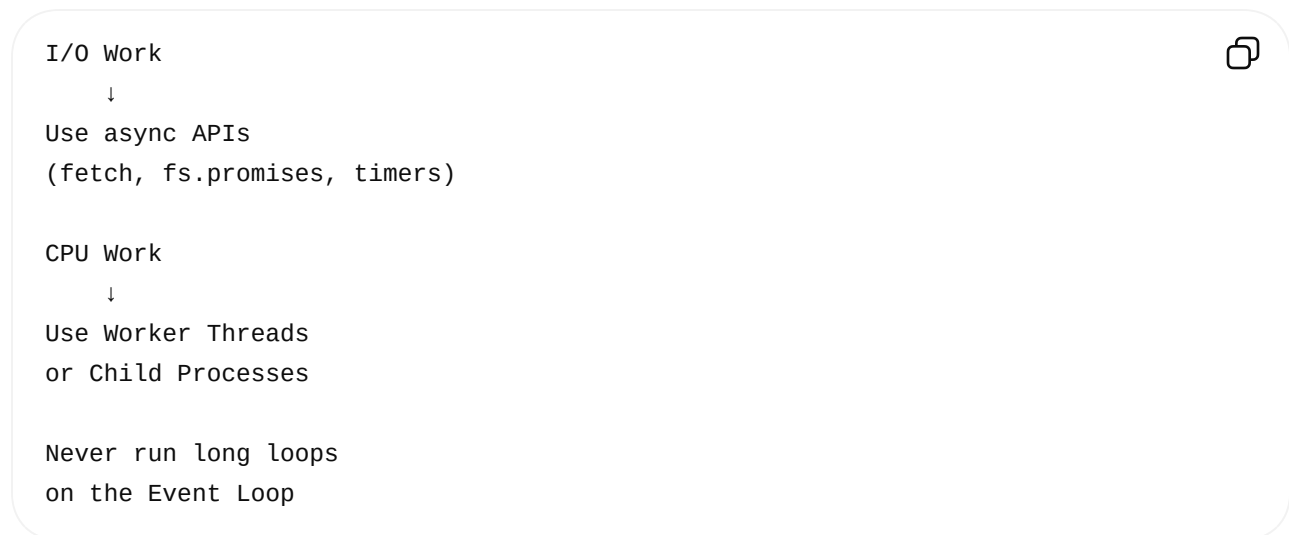


Summary Table

Method	Blocking?	Where Work Happens
while(...)	Yes	Event Loop
for(...) huge loop	Yes	Event Loop
JSON.stringify()	Yes	Event Loop
JSON.parse()	Yes	Event Loop
setTimeout()	No	Timer system
setImmediate()	No	Event Loop queue
fetch()	No	OS networking

Method	Blocking?	Where Work Happens
<code>fs.promises.readFile()</code>	No	libuv thread pool
<code>crypto.pbkdf2()</code>	No	libuv thread pool
<code>crypto.pbkdf2Sync()</code>	Yes	Event Loop
<code>Worker</code>	No	Separate thread
<code>fork()</code>	No	Separate process

A useful mental model is:



That's the core distinction Node.js developers learn when building high-performance servers.

